

# 智慧社区巡检机器人

复赛答辩展示

参赛队伍 WHC

参赛赛项 第八届全国校园人工智能算法精英大赛·算法应用赛（赛马制）·智慧社区

参赛阶段 复赛

日期 2026 年 10 月

# 汇报提纲

赛题分析与总体思路

系统总体架构与技术选型

关键算法设计（建图 / 定位 / 导航 / 调度）

★ 遇到的核心困难与解决方案

实测数据与效果

系统创新点与工程亮点

**Sim2Real** 部署策略

总结与展望

# 1 · 赛题分析与总体思路

赛项：智慧社区 —— 围绕“人、车、事”的社区精准治理

复赛阶段：纯仿真环境下的完整巡检流程验证（无硬件 / 无实车）

核心技术链路：建图 → 定位 → 规划 → 控制 → 任务调度 → 视觉识别

三层拆解：感知层（建图 / 定位 / 检测）· 决策层（规划 / 调度）· 执行层（跟踪 / 避障）

关键判断：难点不在单个算法的先进性，而在“把整条链路在真实约束下稳定跑通”，且具备向实车迁移的能力

## 2 · 系统总体架构与技术选型

| 层次          | 选型                         | 说明                |
|-------------|----------------------------|-------------------|
| 操作系统        | Ubuntu 24.04 (WSL2)        | 开发环境              |
| 中间件         | ROS 2 Jazzy                | DDS 通信，长期支持       |
| 仿真器         | Gazebo Harmonic (gz-sim 8) | 经 ros_gz 桥接 ROS 2 |
| <b>SLAM</b> | slam_toolbox               | 图优化 + 回环检测        |
| 定位          | AMCL                       | 已知地图粒子滤波          |
| 导航          | Nav2                       | 行为树驱动             |
| 全局规划        | SmacPlanner2D              | A* 改进 + 代价惩罚      |
| 局部控制        | Regulated Pure Pursuit     | 几何纯追踪 + 速度调节      |
| 调度          | community_patrol (自研)      | 多点巡检编排            |
| 视觉          | YOLO + OCR (视觉组)           | 6 类目标检测 + 车牌      |

### 3 · 关键技术链路

- ① 仿真场景：**4.2 m × 4.2 m** 场地，围墙 + 车道 + 红绿灯 + 人偶立牌 + 车牌车
- ② 建图：**slam\_toolbox** 在线建图 → **0.01 m** 栅格地图（**425×426 = 18.1** 万栅格）
- ③ 定位：**AMCL** 粒子滤波，一键启动自动初始位姿
- ④ 规划：**SmacPlanner2D** 全局路径 + 代价惩罚远离车道边缘
- ⑤ 控制：**RPP** 轨迹跟踪 + 三层代价地图避障
- ⑥ 调度：**community\_patrol** 节点编排 **11** 个站点（识别 / 红绿灯 / 车牌）
- ⑦ 通信：按任务特性分别用 **Topic / Service / Action** 三类机制

## 4 · 遇到的核心困难总览

| # | 困难                        | 解决方案要点                             |
|---|---------------------------|------------------------------------|
| 1 | 2D 激光看不到地面车道线             | 三层递进：代价惩罚 + 衰减加速 + Keepout 掩膜      |
| 2 | 默认膨胀半径 0.55 m 堵死 0.8 m 窄道 | 缩至 0.20 m + 代价快速衰减                 |
| 3 | AMCL 默认需鼠标点初始位姿（违规）       | set_initial_pose 写死 + 固定出生点        |
| 4 | 建图默认参数在小场地错位 / 鬼影         | travel 0.08 / range 6.0 / 分辨率 0.01 |
| 5 | 恢复行为在窄道“越恢复越糟”            | 按比例缩小后退与旋转参数                       |
| 6 | AMCL 感知混淆（对称街区）           | 小扰动自愈，大扰动 set_initial_pose 兜底      |
| 7 | 红绿灯逻辑与导航框架耦合风险            | 不改行为树，自研调度节点解耦                     |
| 8 | tl_1 站点导航反复失败             | 系统化排查出测试残留障碍 → 删除                  |

# 困难 1 · 车道线合规（最大技术难点）

本质约束：赛题要求“不得压越车道线”，但车道线是地板纹理

**2D** 激光扫描平面在地面上方，只能感知立体障碍物，感知不到地面标线

→ 代价地图中不存在车道线约束，规划器会直接“抄近路”压线

三层递进方案：

- ① 提高代价惩罚  $\text{cost\_penalty} = 5.0$ （主动远离边缘）
- ② 加速代价衰减  $\text{cost\_scaling\_factor} = 6.0$ （车道中心保持低代价）
- ③ 显式禁区掩膜 `KeepoutFilter` + 手工掩膜图（硬约束，脚本已就绪）

## 困难 2 · 膨胀半径堵死窄车道

**Nav2** 默认 **`inflation_radius = 0.55 m`**，而车道宽度仅 **`0.8~1.0 m`**

→ 车道两侧膨胀区相互连成一片，整条车道变为全域高代价

→ 规划器直接报 **`Failed to create plan`**，系统完全无法工作

解决：膨胀半径 **`0.55 → 0.20 m`**（机器人半径 **`0.12 m`** + **`8 cm`** 安全余量）

同时 **`cost_scaling_factor 3.0 → 6.0`**，代价从障碍物向外快速衰减

→ 保证车道中心始终保持低代价，路径自然走在车道中间

## 困难 3 · 恢复行为顶墙 + 速度过快冲线

恢复行为：**Nav2** 默认后退 **0.3 m** / 旋转 **90°**，针对中等尺度设计

→ 在 **0.8 m** 窄道中“越恢复越糟”（退到另一侧墙、转出车道）

解决：按比例缩小恢复行为的位移与角度参数

速度：默认 **0.5 m/s** 在小场地会冲过停止线、过弯甩出车道

解决：**desired\_linear\_vel = 0.18 m/s**，配合到位精度 **xy 0.08 m / yaw 0.12 rad**

# 困难 4 · 建图参数在小场地失效

| 参数                      | 默认      | 本项目      | 调整理由               |
|-------------------------|---------|----------|--------------------|
| minimum_travel_distance | 0.5 m   | 0.08 m   | 4.2 m 场地转弯/墙角栅格错位  |
| minimum_travel_heading  | 0.5 rad | 0.08 rad | 旋转方向加密匹配           |
| max_laser_range         | 30 m    | 6.0 m    | 匹配雷达量程，挡掉穿墙/多径“鬼影” |
| resolution              | 0.05 m  | 0.01 m   | 1 cm 栅格清晰还原窄车道与薄墙  |
| map_update_interval     | 5 s     | 1 s      | 边走边出图更流畅           |

## 困难 5 · “一键启动、不得人为干预”合规

赛题要求：一键启动、自主执行，除严重故障不得人为干预

问题：Nav2 的 AMCL 默认需在 RViz 用鼠标点 2D Pose Estimate 指定初始位姿

→ 这本身即构成人为干预（违规操作）

解决：从配置层面消除 ——

开启 set\_initial\_pose: true 并写死初始位姿

配合 launch 文件固定机器人出生点，每次运行初值一致

## 困难 6 · AMCL 感知混淆（场景对称性）

| 扰动幅度                                | 恢复结果              | 定位误差    |
|-------------------------------------|-------------------|---------|
| 基线（无扰动）                             | —                 | ~2 cm   |
| <b>0.42 m</b> 小扰动                   | 3 s 内自主收敛         | ~1.1 cm |
| <b>1.5 m</b> 大扰动（等价 <b>kidnap</b> ） | 锁定错误对称位置，21 s 未自愈 | ~1.25 m |

结论：小噪声/漂移 AMCL 可自行纠正；大扰动（对称街区导致感知混淆）需 `set_initial_pose` 兜底。

## 困难 7 · 红绿灯逻辑与导航框架解耦

若在 **Nav2** 行为树中实现红绿灯，会与上游版本强耦合、升级易失效、调试成本高

解决：不改动行为树，业务逻辑集中在自研 **community\_patrol** 调度节点

→ 导航框架保持标准，业务逻辑集中可控（高内聚低耦合）

隐蔽 **bug**：等待导航结果时若不持续 **spin\_once**，订阅回调不触发

→ 会导致导航途中收不到红绿灯状态，红灯停车逻辑完全失效

→ 解决：等待循环中持续调用 **rclpy.spin\_once(timeout\_sec=0.1)**

## 困难 8 · tl\_1 站点导航失败的系统化排查

现象：整圈巡检只有 **tl\_1**（红绿灯 1 停止线）**3** 次超时/中止，其余 **10** 站全到达

排查链（**4** 步定位根因）：

- ① 代价地图：目标点 (-0.1,1.3) 为 OCC（占用），停止线处 FREE
- ② 静态地图：该点像素 FREE → 排除建图残留
- ③ gz model --list：发现遗留的 obstacle\_test 障碍块（Q11 测试残留）
- ④ 删除该障碍 → 代价地图清零 → 单点 SUCCEEDED，整圈 11/11 全通过

结论：非代码 **bug**，是测试残留物 —— 体现“数据→地图→场景”逐层定位的方法论

## 5 · 实测数据与效果 ( headless 仿真 )

| 指标              | 实测结果                                      |
|-----------------|-------------------------------------------|
| 巡检一圈通过率         | 11 / 11 站全部到达 , 0 失败                      |
| 一圈总耗时           | 156 s ( 仿真时钟 , 含 6 个识别站各停 3 s + 2 个红绿灯站 ) |
| 规划耗时 <b>Q8</b>  | 0.4 ~ 3.6 ms ( << 100 ms 达标 )             |
| 动态避障 <b>Q11</b> | 障碍出现 0.51 s 触发重规划 , 绕行最近距离 30.4 cm        |
| 定位恢复 <b>Q5</b>  | 基线 ~2 cm ; 0.42 m 扰动 3 s 收敛到 1.1 cm       |

## 6 · 系统创新点与工程亮点

- ① 设计阶段即识别“**2D** 激光看不到地面标线”的本质约束，三层递进解法
- ② 配置层面解决“一键启动”合规性，消除人工点位姿
- ③ 导航框架与业务逻辑解耦，不修改 **Nav2** 行为树
- ④ 面向 **4.2 m** 小尺度场地的系统性参数标定（多处默认值失效的逐一修正）
- ⑤ 完整运用 **Topic / Service / Action** 三类通信机制

## 7 · Sim2Real 部署策略

- ① 参数外置：所有参数集中 **YAML**，实车只换参数文件，不改一行代码
- ② 噪声注入：主动注入激光噪声 / 里程计漂移 / 光照变化 / 动态障碍，提前暴露鲁棒性问题
- ③ 接口抽象：调度层只依赖 **Nav2** 标准 **Action**，不依赖具体底盘，上层逻辑完全复用

实测鲁棒性：**0.42 m** 扰动 **AMCL 3 s** 自愈到 **1.1 cm**；大扰动需 **set\_initial\_pose** 兜底

## 8 · 总结与展望

完整链路：场景构建 → **SLAM** 建图 → 精确定位 → 全局规划 → 局部控制 → 多点调度 → 视觉联动

价值：不仅算法实现完整，更在于对赛题约束的深入分析与工程化设计

面向总决赛：

- ① 完成仿真到实车的算法迁移与参数标定
- ② 扩展识别任务（垃圾桶 / 楼宇火灾 / 异常温度 / 仪表 / 电动车状态）
- ③ 针对现场 5 分钟限时优化调度策略与运动效率

谢谢各位评委老师！